

Agent Manifest: Portable, Hardware-Attestable Identity and Provenance for AI Agents

Imran Siddique
OPAQUE Systems

<https://github.com/agenttrust-io/agent-manifest>

October 1, 2026
Technical report, version 2

Abstract

Agent Manifest proposes a signed record of the artifacts that define an AI agent, including its system prompt, model, policy, tools, retrieval corpus, memory, decision trace, delegation, supply chain, and human approvals. Artifact digests and canonical serialization make changes to the recorded baseline detectable when a verifier has the relevant artifacts and trusted keys. The design composes SPIFFE identities, RFC 8785 canonicalization, Ed25519 signatures, an optional ML-DSA-65 profile, and hardware-attestation bindings. This technical report describes the original model and preserves the recorded Ed25519 software benchmarks, delegation checks, and tamper probes. Signed metadata alone does not prove that the recorded configuration was used during execution. The evaluation does not measure post-quantum performance or establish live hardware provenance; continuous verification of mutable memory and retrieval state is outside its scope. Current SDK behavior and normative requirements are maintained separately.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. It revises a manuscript originally dated June 23, 2026; the recorded experiments include later runs and have not been rerun for this edition. Architectural requirements describe intended checks, not proof that the historical SDK implements every check. Consult the linked project specification and limitations before implementing the design. Version 2 (October 1, 2026) corrects the signature pre-image description, the tamper-probe field list, the delegation per-hop figure and the conformance test count, states the inferred SDK revision, records that the evaluated verifier did not authenticate human approvals, adds the result of an October 2026 rerun, replaces two repository citations with the published reports, and restores bold and italic type that the Tectonic build had dropped; the recorded numbers are unchanged.

Patent Pending.

Contents

1	Introduction	3
2	Background	3
3	The Manifest	4
3.1	The ten artifact bindings	4
4	Cryptographic Protocols	6
4.1	Canonicalization and signing	6
4.2	Merkle commitments	6
4.3	Post-quantum profile	6
5	Verification	6
5.1	Signed fields	6

5.2	Fail-closed semantics	6
5.3	Revocation	7
6	Delegation and Human Oversight	7
6.1	Delegation chains	7
6.2	Human-in-the-loop approvals	7
7	Integration	7
8	Conformance and Reproducibility	8
8.1	Conformance levels	8
8.2	Conformance modules	8
8.3	Evaluation	8
9	Related Work and Regulatory Mapping	10
10	Conclusion	10

1 Introduction

An AI agent is not a single artifact. What it does is determined by a system prompt, a policy bundle, a set of tool schemas, a model and its version, a retrieval corpus, a memory state, the decisions it records, the sub-agents it delegates to, its supply chain, and the human approvals that gate its actions. Today none of these is attested. A signed JWT proves which credential called an API; it proves nothing about the agent behind the call.

This is the *agent identity gap*. When an agent acts in a regulated environment, its operator can show policy documents, SOC reports, and mutable application logs, none of which prove what actually ran. A reviewer cannot confirm which system prompt defined the agent, which model version executed, which policy was in force, which tools were authorized, whether the retrieval corpus or memory was poisoned, what the agent decided, whether a human approved a high-risk action, or whether the deployed binary matches the one reviewed.

Agent Manifest closes this gap. It binds the artifacts that define an agent into one cryptographically signed, hardware-attestable, tamper-evident record. Its central design choice is *tamper-evidence over tamper-resistance*: rather than trying to prevent change, the manifest makes change *detectable* by any third party that holds it and compares it to hardware attestation. Each artifact is bound by hash; the manifest is canonicalized with RFC 8785 [Rundgren et al., 2020], signed with Ed25519 [Josefsson and Liusvaara, 2017] (or post-quantum ML-DSA-65 [National Institute of Standards and Technology, 2024]), and registered on an append-only transparency log.

The ten unattested surfaces

Agent Manifest binds ten artifacts: (1) the system prompt, (2) the policy bundle, (3) the tool manifest, (4) the model identity, (5) the retrieval (RAG) corpus, (6) the memory baseline, (7) the decision trace, (8) the A2A delegation chain, (9) the supply chain, and (10) the human-in-the-loop approvals. Each is a surface an adversary can alter without detection in today’s deployments; each becomes tamper-evident once bound (Section 3).

Contributions

- **A manifest schema** binding ten agent-defining artifacts, with a SPIFFE identity and a confirmation key (Section 3).
- **Cryptographic protocols**: RFC 8785 canonicalization, Ed25519 and a post-quantum ML-DSA-65 profile, and RFC 9162 Merkle commitments (Section 4).
- **A fail-closed verification protocol** with explicit result states and revocation (Section 5).
- **Delegation and human-oversight** bindings that prevent scope laundering across agent hops (Section 6).
- **Integration and conformance**: composition with cMCP, TRACE, and AGT (Section 7), and an open SDK with a conformance specification of 197 tests across four levels (Section 8).

Paper structure

Section 2 reviews the primitives Agent Manifest composes. Section 3 defines the manifest. Section 4 gives the cryptographic protocols and Section 5 the verification protocol. Section 6 covers delegation and human oversight, Section 7 integration with the surrounding stack, Section 8 conformance, Section 9 related work and regulatory mapping, and Section 10 concludes.

2 Background

Agent Manifest composes established primitives rather than inventing new ones.

Workload identity. SPIFFE [SPIFFE Project, 2018] gives workloads cryptographic identities of the form `spiffe://trust-domain/path`. Agent Manifest uses SPIFFE URIs as the canonical form for both `agent_id` (the agent) and `issuer` (the signing authority), reusing identity already issued by SPIRE, Kubernetes, and service meshes

without a new registry. Human approvers, who are not workloads, are named instead by OIDC subject, `mailto:` URI, or W3C DID.

Attestation. The manifest’s hardware evidence follows RATS [Birkholz et al., 2023] and normalizes platform reports through EAT [Lundblade et al., 2025]. A TEE attestation report binds the manifest’s signing key to a measured environment, so a verifier can confirm the manifest was produced where it claims.

Canonicalization and signatures. Every signature is computed over the RFC 8785 JSON Canonicalization Scheme (JCS) [Rundgren et al., 2020] form of the signed fields, so independent implementations produce byte-identical pre-images. Signatures use Ed25519 [Josefsson and Liusvaara, 2017] in the standard profile and ML-DSA-65 [National Institute of Standards and Technology, 2024] in the post-quantum profile.

Tamper-evident logs. Artifact collections (the retrieval corpus, the decision trace) and the transparency anchor use Merkle trees with RFC 9162 [Laurie et al., 2021] domain-separated interior nodes; transparency-log entries follow the Sigstore/Rekor model [Newman et al., 2022]. Identifiers are time-ordered UUIDv7 [Davis et al., 2024].

Relationship to the runtime stack. Agent Manifest answers *who and what* an agent is. It is complementary to cMCP [Poddar et al., 2026a], which enforces policy and produces hardware-attested evidence at the tool-call boundary, and to TRACE [Poddar et al., 2026b], the portable evidence format cMCP emits; the manifest’s decision-trace artifact references a TRACE audit chain. It composes with the Agent Governance Toolkit (AGT) [Microsoft Corporation, 2026], which supplies policy enforcement, trust scoring, and observability: the manifest’s attestation level feeds AGT’s trust score, its delegation chain feeds scope-narrowing policy, and its revocation feeds AGT’s kill switch.

3 The Manifest

A manifest is a JSON document with a canonical (RFC 8785) serialization over which a signature is computed. Table 1 lists the top-level fields.

Table 1: Top-level manifest fields (optional fields italicized).

Field	Type	Notes
<code>manifest_id</code>	UUIDv7 [Davis et al., 2024]	time-ordered unique id
<code>agent_id</code>	SPIFFE URI	the agent workload
<code>version</code>	string	schema version (0.1)
<code>issued_at</code>	ISO 8601 UTC	issuance time
<code>expires_at</code>	ISO 8601 UTC	≥ 1 h after, ≤ 365 days
<code>issuer</code>	SPIFFE URI	signing authority
<code>crypto_profile</code>	standard post-quantum	signature suite
<code>artifacts</code>	object	the ten artifact bindings (Table 2)
<code>attestation</code>	object	TEE evidence; required at Level 1+
<code>delegation_chain</code>	list	present when the agent was delegated to
<code>hitl_record</code>	object	present when policy mandates human approval
<code>signature</code>	object	algorithm, key id, signed-field list, value
<code>transparency_log_entry</code>	object	inclusion proof; required in production

3.1 The ten artifact bindings

The `artifacts` object binds each agent-defining surface by hash and metadata, so any post-approval change to the underlying artifact is detectable. Table 2 summarizes the bindings.

Table 2: The ten artifact bindings and their key bound fields.

#	Binding	Key bound fields
1	System prompt	hash, version, classification, safety level
2	Policy bundle	hash, policy language (Cedar/Rego/YAML/composite), enforcement mode, AGT version
3	Tool manifest	catalog_hash, per-tool entries, dynamic-registration and rug-pull policy
4	Model identity	provider, model id, version, deployment type, model_hash, attestation type
5	RAG corpus	merkle_root, document count, ingestion-policy hash, poisoning scan
6	Memory baseline	snapshot_hash, memory type, drift policy, TTL
7	Decision trace	audit_chain_root (a TRACE chain), trace type, signing key id, entry count
8	Delegation chain	per-hop principal, scope_grant, delegation signature, principal manifest id
9	Supply chain	container/base-image digests, SLSA provenance, SBOM, MCP servers
10	Human-in-the-loop	required flag, approvals, escalation policy

Figure 1 shows the binding structure: ten heterogeneous artifacts each reduce to a digest, the digests collect into the `artifacts` object, the whole document is serialized with RFC 8785 canonical JSON, and one signature plus an optional transparency-log inclusion proof close the record. Any change to any bound artifact changes its digest, changes the canonical bytes, and invalidates the single signature: this is what makes the record tamper-evident.

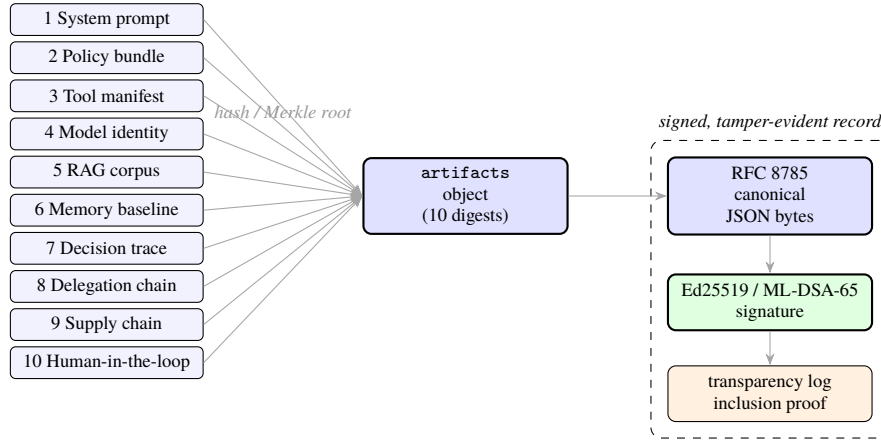


Figure 1: Ten agent-defining artifacts each reduce to a digest, collect into the `artifacts` object, canonicalize under RFC 8785, and are closed by one signature and an optional transparency-log inclusion proof. Any post-approval change to any artifact changes its digest and invalidates the single signature.

The binding pattern is uniform: each artifact contributes a digest (a SHA-256 hash, a Merkle root for collections, or an audit-chain root for the decision trace) into the signed manifest, so verifying the signature plus recomputing the artifact’s digest detects any divergence. The model-identity binding distinguishes a *hash-bound* model (weights digest verifiable) from a *provider-asserted* one (a hosted API, where the provider’s assertion is the strongest available evidence). The decision-trace binding is the join point with the runtime stack: its `audit_chain_root` is the root of the TRACE audit chain cMCP produces [Poddar et al., 2026b,a].

4 Cryptographic Protocols

4.1 Canonicalization and signing

The signature pre-image is the RFC 8785 JCS [Rundgren et al., 2020] serialization of the manifest’s signed fields, encoded as UTF-8 without a BOM; Ed25519 and ML-DSA-65 sign these bytes directly. SHA-256 (SHAKE-256 in the post-quantum profile) is the artifact-digest and Merkle hash. JCS is required precisely because ad-hoc sorting diverges on non-ASCII strings and IEEE 754 numbers; two conformant implementations must produce identical bytes.

The standard profile signs with Ed25519 [Josefsson and Liusvaara, 2017]; the post-quantum profile signs with ML-DSA-65 (NIST FIPS 204) [National Institute of Standards and Technology, 2024]; the hybrid mode carries both a classical and a post-quantum signature over the *identical* pre-image, and a verifier requires both to validate. A key id is the SHA-256 hex digest of the raw public-key bytes, and signature values are base64url without padding. The signature block records its algorithm, key_id, key_type (software | hsm | tee-sealed), and the explicit list of signed_fields.

4.2 Merkle commitments

Artifact collections commit via Merkle trees with RFC 9162 [Laurie et al., 2021] domain-separated nodes (leaf 0x00, interior 0x01 || left || right), so the retrieval corpus binds as a single merkle_root over its documents and the transparency-log entry carries an RFC 9162 inclusion proof. The decision trace binds as a hash-chained or Merkle-log audit root.

4.3 Post-quantum profile

Because manifests may need to remain verifiable for years, Agent Manifest specifies a post-quantum profile from v0.1: crypto_profile: post-quantum selects ML-DSA-65 and SHAKE-256, and the hybrid mode lets a deployment carry both classical and post-quantum signatures during migration, so a manifest stays verifiable whether or not a verifier has post-quantum support.

5 Verification

5.1 Signed fields

A fixed, normative set of fields is covered by the signature: the manifest metadata (manifest_id, agent_id, version, issued_at, expires_at, issuer, crypto_profile), the artifacts, the delegation_chain, and the hitl_record, among others. Three fields are deliberately *outside* the signature because they are appended after signing: the attestation report (bound instead through the key the report attests), the signature block itself, and the transparency_log_entry (which can only exist after the signed manifest is registered). Within hitl_record, the approvals list is normalized to empty before signing, so approvals attach after issuance and each must be authenticated by its own approval_signature.

5.2 Fail-closed semantics

Verification is fail-closed: VALID requires a verifiable signature against a trusted key. A signed manifest whose key is not trusted returns UNVERIFIABLE; an unsigned manifest returns SIGNATURE_MISSING; neither returns VALID. The verifier emits a structured result: an overall state (VALID, MISMATCH, EXPIRED, REVOKED, INCOMPLETE, ATTESTATION_UNAVAILABLE, INCOMPATIBLE_VERSION, SIGNATURE_MISSING, or UNVERIFIABLE) plus per-artifact field results (MATCH, MISMATCH, NOT_BOUND, EXPIRED) and, where present, delegation and HITL results. A verifier checks version before anything else: an unsupported version returns INCOMPATIBLE_VERSION, never VALID.

5.3 Revocation

A manifest can be revoked before `expires_at` (for a leaked key, a withdrawn approval, or a compromised artifact). Revocation records are signed and distributed as an append-only, JSON-lines certificate revocation list; a verifier consults current revocation state at verification time and returns `REVOKED` for a listed manifest. Revocation also drives the downstream kill switch when Agent Manifest is paired with AGT (Section 7).

6 Delegation and Human Oversight

6.1 Delegation chains

When one agent delegates to another (an A2A hop), the sub-agent’s manifest carries a `delegation_chain`: an ordered list of hops, each naming the delegating principal, a `scope_grant` (the tools and data classifications the hop is permitted), a delegation signature, and the principal’s manifest id and attestation hash. Because each hop is signed and references the parent manifest, a verifier reconstructs the full chain of authority from the leaf agent back to the root.

The chain enforces *monotonic narrowing*: a hop’s scope must be a subset of the scope it was granted, so a sub-agent cannot acquire capabilities its delegator lacked: the *scope-laundering* attack. A maximum delegation depth (default 3) bounds the chain and prevents unbounded re-delegation. The delegation chain is the input AGT uses to apply scope-narrowing policy at runtime (Section 7). Figure 2 shows a three-principal chain and the capability set shrinking at each hop.

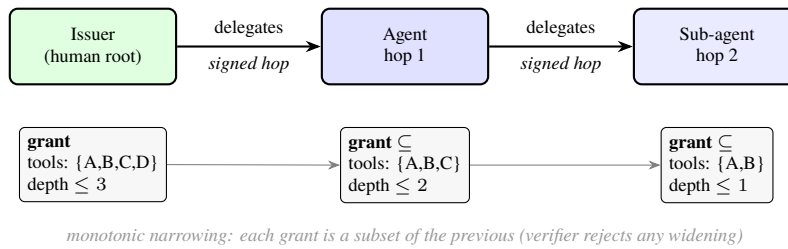


Figure 2: Delegation chain from issuer to agent to sub-agent. Each hop is signed by the delegating principal and references the parent manifest; the capability grant narrows monotonically (tools drop, depth budget decrements). The verifier rejects any hop whose scope exceeds its delegator’s grant, defeating scope laundering.

6.2 Human-in-the-loop approvals

When policy requires a human to authorize an action, the manifest carries an `hitl_record`: the requirement flag, the approvals collected, and an escalation policy. Each approval names the approver (by OIDC subject, `mailto:` URI, or DID, not a SPIFFE workload identity, since the approver is a person), the approved scope (a risk tier and the artifact set the human signed off on), and a timestamp. A verifier returns `APPROVED`, `EXPIRED`, `NOT_REQUIRED`, `MISSING`, or `APPROVAL_INSUFFICIENT`. When each approval’s signature is verified against the approver’s key, an auditor has evidence that an identified human approved the recorded scope, which supports human-oversight obligations such as EU AI Act Article 14 [European Union, 2024]. At the evaluated SDK revision, `verify_manifest` did not check `approval_signature`: a signed manifest whose approvals were replaced with forged ones still returned `VALID` and `APPROVED` (experiments/results/rerun-2026-10-01). The SDK has verified approval signatures in that path since commit 36d0c89 (August 22, 2026).

7 Integration

Agent Manifest is deliberately one layer of a stack. It answers *identity and provenance*; three neighboring systems answer the rest.

cMCP: runtime attestation. cMCP [Poddar et al., 2026a] is the reference runtime that enforces the manifest’s policy and tool bindings at the Model Context Protocol boundary. It detects tool-catalog “rug-pulls” before the agent sees the change, and it signs a decision record per tool call. The manifest’s decision-trace artifact (Table 2, #7) binds the root of the audit chain cMCP produces, so a verifier can tie the static manifest to the dynamic record of what the agent did.

TRACE: portable evidence. TRACE [Poddar et al., 2026b] is the portable, hardware-attested evidence format cMCP emits for each session. Where the manifest establishes the agent at deployment time, a TRACE Trust Record attests its behavior at run time; the two are joined by the manifest’s `audit_chain_root`, which is the same chain TRACE commits.

AGT: policy and observability. The Agent Governance Toolkit [Microsoft Corporation, 2026] consumes the manifest at runtime: the attestation level feeds AGT’s trust score, the delegation chain feeds scope-narrowing policy, the decision-trace root feeds AGT’s governance event sink, and a revocation record triggers AGT’s kill switch. Agent Manifest supplies the verifiable facts; AGT acts on them.

MCP and A2A. The tool-manifest binding extends the MCP [Model Context Protocol Project, 2024] initialize handshake with a pinned catalog hash, and the delegation chain proposes an A2A delegation format. Both are protocol-agnostic; MCP is the reference surface.

8 Conformance and Reproducibility

Agent Manifest is realized by an open-source Python SDK (`agent-manifest`) and a formal conformance suite. The SDK provides the data model, signing (`generate_ed25519`, signers/verifiers), verification (`verify_manifest`), canonicalization, Merkle construction, delegation and revocation, hardware-attestation providers (TPM, SEV-SNP, TDX, OPAQUE) with auto-selection, and a manifest CLI (`keygen`, `create`, `sign`, `attest`, `verify`, `revoke`).

8.1 Conformance levels

Conformance is defined at four assurance levels, each adding evidence (Table 3). A claim of conformance is checked, not asserted. The suite runs against versioned example manifests: software-only, TPM-attested, a fully populated ten-artifact manifest, multi-hop delegation chains, a HITL manifest, and a revocation record.

Table 3: Agent Manifest conformance levels.

Level	Adds	Use case
0	signature + binding validation (software keys)	development and staging
1	TPM attestation	production with a TPM root of trust
2	SEV-SNP / TDX attestation	confidential-VM deployments
3	post-quantum (ML-DSA-65) signatures	long-lived / PQ-required records

8.2 Conformance modules

The specification defines 197 conformance tests across five modules (Table 4). At the evaluated SDK revision the five corresponding test files contain 191 tests (AM-BIND 44, AM-CRYPTO 29, AM-ATTEST 38, AM-VERIFY 49, AM-COMPAT 31; four post-quantum tests skip without the ML-DSA dependency), within an SDK suite of 558 tests covering binding, canonicalization, signing, attestation, delegation, HITL, revocation, transparency, Merkle construction, and corpus poisoning detection.

8.3 Evaluation

We measured signing and verification cost, on-the-wire size, delegation-chain scaling, and tamper detection with a deterministic harness (`experiments/bench.py`) that drives the shipped SDK directly. The manifests timed are the

Table 4: Conformance modules and the test counts the specification defines (197 total).

Module	Validates	Tests
AM-BIND	artifact binding, hashing, Merkle trees	47
AM-CRYPTO	signatures and RFC 8785 canonicalization	38
AM-ATTEST	TEE attestation binding and field cross-checks	29
AM-VERIFY	verification, mismatch, delegation, revocation	52
AM-COMPAT	AGT, cMCP, and MCP integration	31
Total		197

SDK’s own committed example manifests (loaded, stripped of their placeholder signatures, re-signed with a fresh key, and confirmed to verify VALID before timing) so every input is schema-valid under the shipped Pydantic models and no manifest body is hand-authored. All numbers below are medians over 2000 iterations (1000 for delegation chains) after 50 warmup iterations, on Python 3.12, Windows 11, ARMv8 (Qualcomm). The SDK revision was not recorded; the inferred revision is agent-manifest 67c21db, whose SDK source and examples match tag `python-v0.2.0`. Only the Ed25519 *standard* profile is measured: the post-quantum ML-DSA-65 profile requires Open Quantum Safe bindings (`import oqs`), which are not installed in this environment, and the SDK’s pq extra names `pyoqs`, which is not published on PyPI, so no PQ figures are reported.

Table 5 reports per-manifest sign/verify latency and size. Signing a manifest costs under 2 ms and verification under 2.7 ms across the full range, from a 4-artifact Level 0 manifest (2.0 KiB canonical) to a Level 1 manifest with six artifact bindings plus TEE attestation and a transparency-log entry (4.9 KiB canonical). Figure 3(a) plots the sign/verify medians and Figure 3(b) the canonical (JCS) sizes. A rerun on October 1, 2026 at the inferred revision, with the SDK clock frozen at the original run time because the example manifests expired in September 2026, reproduced every size and tamper outcome; its Level 1 verify median was 3.9 to 4.9 ms over three runs, so the 2.7 ms bound did not reproduce (experiments/results/rerun-2026-10-01).

Table 5: Ed25519 sign / verify latency (median and p95, ms) and size for the committed example manifests. Real measured output of `experiments/bench.py`.

Manifest	Artifacts	Sign med.	Sign p95	Verify med.	Verify p95	JCS bytes
Level 0 (software)	4	0.57	1.18	1.16	2.08	2018
Multi-artifact (+RAG)	5	1.46	2.70	1.55	2.29	2968
HITL (+approval)	4	0.71	1.20	1.18	1.95	2311
Level 1 (+attest, +tlog)	6	1.97	3.02	2.66	3.80	4968

Delegation-chain scaling. We built signed delegation chains of depth 1 to 10 using the shipped `DelegationHopSigner`, narrowing scope monotonically at each hop (dropping one tool and decrementing the depth budget), and verified each chain with the shipped `verify_delegation_chain`. Verification latency is close to linear in chain depth (Figure 4): depths 1 to 9 cost about 0.21 ms per hop (0.21 ms for one hop, 1.99 ms for nine), and the ten-hop median, 2.96 ms, sits above that line.

Tamper detection. Starting from a signed Level 1 manifest that verifies VALID, we mutated each of ten bound fields in turn (six artifact fields, namely five digests and the model version, plus `agent_id`, `issuer`, `manifest_id`, and `issued_at`) and re-verified without re-signing. In all ten cases verification no longer returned VALID: the mutated canonical bytes fail the single signature, so the tamper is caught (fail-closed). This shows that the signature covers these fields. The probe supplies no runtime artifact digests, so comparison against a deployed agent’s artifacts is not exercised. The verifier at this revision also did not recompute the declared tool-catalog root from the per-tool entries: the committed Level 0 example declares a root that its own tool list does not produce, verifies VALID here, and returns MISMATCH on current SDK main.

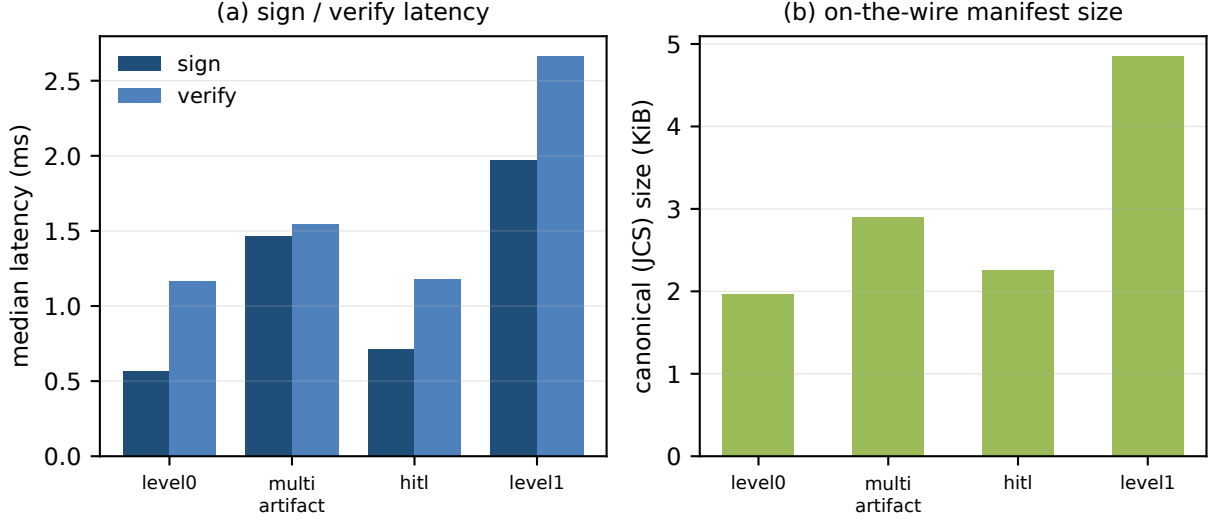


Figure 3: (a) Median Ed25519 sign and verify latency and (b) canonical (JCS) manifest size across the committed example manifests. Generated by `experiments/plot.py` from the measured `bench.json`.

9 Related Work and Regulatory Mapping

Standards composition. Agent Manifest composes rather than replaces. It reuses SPIFFE [SPIFFE Project, 2018] for identity, RFC 8785 [Rundgren et al., 2020] for canonicalization, Ed25519 [Josefsson and Liusvaara, 2017] and ML-DSA-65 [National Institute of Standards and Technology, 2024] for signatures, RFC 9162 [Laurie et al., 2021] Merkle trees and Sigstore/Rekor [Newman et al., 2022] for transparency, SLSA [OpenSSF, 2023] and SBOM formats for supply-chain provenance, RATS/EAT [Birkholz et al., 2023, Lundblade et al., 2025] for attestation, and UUIDv7 [Davis et al., 2024] for identifiers. Its contribution is the *binding rule* that places each of these into one signed, hardware-attestable record scoped to an agent.

Relative to the runtime stack. Static documentation (model cards, AIBOMs) describes intent but is unbound to what deployed; runtime governance frameworks enforce behavior but do not attest identity and provenance; platform attestation proves the environment but not the agent. Agent Manifest is the identity-and-provenance layer that the runtime evidence formats (TRACE and cMCP [Poddar et al., 2026b,a]) and governance toolkits like AGT [Microsoft Corporation, 2026] build on.

Regulatory mapping. The manifest’s records map onto emerging obligations: tamper-evident record-keeping for high-risk AI (EU AI Act Article 12) is served by the signed, transparency-anchored manifest; human-oversight requirements (Article 14) [European Union, 2024] by the HITL record (Section 6); and equivalent audit-trail duties under sector regimes (financial, healthcare) by the same evidence. Agent Manifest does not claim compliance; it produces the cryptographic evidence a compliance program needs.

10 Conclusion

Agent Manifest gives an AI agent a verifiable identity. By binding the ten artifacts that define an agent (system prompt, policy, tools, model, retrieval corpus, memory, decision trace, delegation, supply chain, and human approvals) into one signed, hardware-attestable, tamper-evident record, it lets a verifier detect differences between supplied artifacts and a signed baseline. Establishing that those artifacts were used in a deployed agent requires additional runtime evidence and trusted attestation binding. The design favors tamper-evidence over tamper-resistance, reuses established primitives (SPIFFE, JCS, Ed25519/ML-DSA, RFC 9162 Merkle trees, RATS/EAT), and slots into the runtime stack alongside

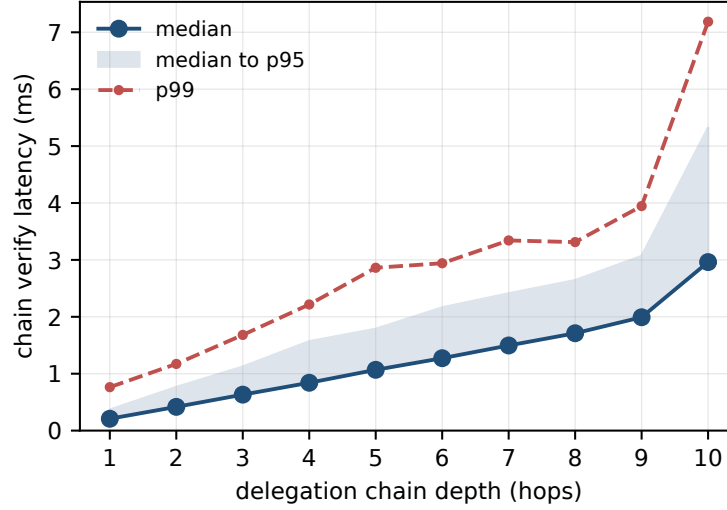


Figure 4: Delegation-chain verification latency versus chain depth (1 to 10 hops), median with the median-to-p95 band and the p99 line. Each hop is signed by the shipped `DelegationHopSigner` and verified by `verify_delegation_chain`.

cMCP and TRACE for attestation and AGT for policy.

Limitations and future work. The format is settled for the static artifacts; several pieces are staged. Memory and retrieval bindings capture a baseline and a drift policy but continuous in-flight verification of mutating state is future work. The post-quantum profile is specified and implemented but was not exercised: its tests skip without the ML-DSA dependency. Federated, cross-organization verification (where one trust domain appraises another’s manifests) is targeted for the next revision, alongside richer multi-model manifests. The specification is a developer-preview draft; the roadmap is community hardening toward an open standard with multiple independent implementations and conformance certification.

References

- Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote attestation procedures (RATS) architecture. RFC 9334, 2023. URL <https://www.rfc-editor.org/rfc/rfc9334>.
- Kyzer Davis, Brad Peabody, and Paul Leach. Universally unique identifiers (UUIDs). RFC 9562, 2024. URL <https://www.rfc-editor.org/rfc/rfc9562>.
- European Union. Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (ai act), 2024. URL <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>. Article 12 (record-keeping), Article 14 (human oversight).
- Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, 2017. URL <https://www.rfc-editor.org/rfc/rfc8032>.
- Ben Laurie, Eran Messeri, and Rob Stradling. Certificate transparency version 2.0. RFC 9162, 2021. URL <https://www.rfc-editor.org/rfc/rfc9162>.
- Laurence Lundblade, Giridhar Mandyam, Jeremy O’Donoghue, and Carl Wallace. The entity attestation token (EAT). RFC 9711, 2025. URL <https://www.rfc-editor.org/rfc/rfc9711>.
- Microsoft Corporation. Agent governance toolkit (AGT), 2026. URL <https://github.com/microsoft/agent-governance-toolkit>. Accessed June 2026.

- Model Context Protocol Project. Model context protocol specification, 2024. URL <https://modelcontextprotocol.io/specification>. Accessed June 2026.
- National Institute of Standards and Technology. Module-lattice-based digital signature standard (ML-DSA). Technical Report FIPS 204, NIST, 2024. URL <https://csrc.nist.gov/pubs/fips/204/final>.
- Zachary Newman, John Meyer, and Santiago Torres-Arias. Sigstore: Software signing for everybody. ACM CCS, 2022. URL <https://dl.acm.org/doi/10.1145/3548606.3560596>.
- OpenSSF. SLSA: Supply-chain levels for software artifacts, v1.0, 2023. URL <https://slsa.dev/spec/v1.0/>. Accessed June 2026.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. cMCP: Verifiable policy enforcement for AI agent tool calls. Technical report, version 1, OPAQUE Systems, 2026a. URL <https://doi.org/10.5281/zenodo.23001694>.
- Rishabh Poddar, Aaron Fulkerson, and Imran Siddique. TRACE: Trust, runtime attestation, and compliance evidence. Technical report, version 2, OPAQUE Systems, 2026b. URL <https://doi.org/10.5281/zenodo.23024103>.
- Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). RFC 8785, 2020. URL <https://www.rfc-editor.org/rfc/rfc8785>.
- SPIFFE Project. SPIFFE: Secure production identity framework for everyone, 2018. URL <https://spiffe.io/docs/latest/spiffe-about/overview/>. Accessed June 2026.